

Git Commands Cheat Sheet



You said:

all git and github command for developer to use

ChatGPT said:

If you try to memorize *all* Git commands, you'll burn time. What actually helps is mastering the **core workflow + a few advanced tools**. I'll give you a clean, developer-focused cheat sheet—from beginner to advanced.

1. Basic Setup (One-time)

```
Bash
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --list
```

2. Repository Setup

```
Bash
git init                # Initialize repo
git clone <repo_url>    # Clone repo
```

3. Basic Workflow (MOST IMPORTANT)

```
Bash
git status          # Check changes
git add file.txt    # Stage file
git add .           # Stage all files
git commit -m "message" # Commit changes
git push           # Push to remote
git pull           # Pull latest changes
```

4. Branching (VERY IMPORTANT)

```
Bash
git branch          # List branches
git branch new-branch # Create branch
git checkout new-branch # Switch branch
git switch new-branch # Modern way
git checkout -b new-branch # Create + switch

git merge branch-name # Merge branch
git branch -d branch-name # Delete branch
```

5. Remote (GitHub)

```
Bash
git remote add origin <url>
git remote -v

git push -u origin main
git push origin branch-name

git pull origin main
git fetch
```

6. Undo / Fix Mistakes (VERY USEFUL)

```
Bash
git restore file.txt          # Undo changes in file
git restore --staged file.txt  # Unstage file

git reset --soft HEAD~1      # Undo commit (keep changes)
git reset --hard HEAD~1      # Undo commit (delete changes)

git revert <commit_id>       # Undo commit safely

git checkout -- file.txt     # Old way restore
```

7. Logs & History

```
Bash
git log
git log --oneline
git log --graph --oneline --all

git diff                      # Show changes
git diff --staged
```

8. Stashing (Temporary Save)

```
Bash
git stash
git stash list
git stash apply
git stash pop
```

9. Merge & Conflict Handling

```
Bash
git merge branch-name

# After conflict
git add .
git commit
```



10. Advanced (Important for Developers)

```
Bash
git rebase main          # Rebase branch
git rebase -i HEAD~3     # Interactive rebase

git cherry-pick <commit_id>

git tag v1.0
git tag -a v1.0 -m "version 1.0"
git push origin v1.0
```



11. GitHub Authentication (Modern)

```
Bash
git remote set-url origin
https://<token>@github.com/user/repo.git
```

OR use SSH:

```
Bash
ssh-keygen -t ed25519 -C "your_email"
```



12. Clean & Maintenance

```
Bash
git clean -fd          # Remove untracked files
git gc                 # Cleanup repo
```



13. Pro Developer Commands

```
Bash
git blame file.txt      # Who changed what
git show <commit_id>    # Show commit details
git shortlog            # Contributors list
```



Real Developer Workflow (Daily Use)

```
Bash
git pull origin main
git checkout -b feature-login
git add .
git commit -m "Added login feature"
git push origin feature-login
```



Important Tips

- Use **branches** for every feature
- Never commit directly to **main**
- Always **pull** before pushing
- Write meaningful commit messages
- Use **.gitignore** properly

If you want next level 🚀

I can:

- Give you **real Git interview questions**
- Create **Git practice scenarios (like real company workflow)**
- Teach you **how to fix ANY Git error (like the one you faced earlier)**

You said:

explain every command to why use and what is work

ChatGPT said:

Trying to understand **every Git command deeply** is the right move—but doing it blindly won't stick. The key is to connect each command to the **problem it solves in real development**.

I'll explain the important commands in a **developer mindset: WHY + WHAT + WHEN to use** 👉

1. git init

```
Bash
git init
```

WHY: Start tracking a project with Git

WHAT IT DOES: Creates a hidden `.git` folder (this is your repo brain)

WHEN: When you create a new project from scratch

👉 Without this, Git doesn't exist in your folder.

2. git clone

```
Bash
git clone <repo_url>
```

WHY: Get code from GitHub (or any remote)

WHAT: Downloads full project + history

WHEN: Joining a project / starting from existing repo

👉 It also automatically connects to remote (`origin`)

3. git status

```
Bash
git status
```

WHY: Know what's happening

WHAT: Shows:

- modified files
- staged files
- untracked files

WHEN: Before every commit (seriously)

👉 This is your "dashboard"

4. `git add`

```
Bash
git add file.txt
git add .
```

WHY: Tell Git *what to include in next commit*

WHAT: Moves files → **staging area**

WHEN: After editing files

👉 Think:

- Working Directory → (git add) → Staging → (git commit) → Repo

5. `git commit`

```
Bash
git commit -m "message"
```

WHY: Save a snapshot

WHAT: Stores staged changes permanently

WHEN: After completing a logical change

👉 Commit = checkpoint in your project

6. `git push`

```
Bash
git push origin main
```

WHY: Upload your code to GitHub

WHAT: Sends commits → remote repo

WHEN: After committing locally

👉 Without push, your code is only on your laptop

7. `git pull`

```
Bash
git pull origin main
```

WHY: Get latest changes from team

WHAT: Fetch + merge

WHEN: Before starting work / before push

👉 Prevents conflicts

8. `git branch`

```
Bash
git branch
git branch feature-login
```

WHY: Work on features separately

WHAT: Creates independent line of development

WHEN: Always for new feature

👉 Keeps `main` clean

9. `git checkout` / `git switch`

```
Bash
git checkout feature-login
git switch feature-login
```

WHY: Move between branches

WHAT: Changes your working directory

👉 `switch` = modern version

10. `git checkout -b`

```
Bash
git checkout -b feature-login
```

WHY: Save time

WHAT: Create + switch branch

👉 Most used command in real dev

11. `git merge`

```
Bash
git merge feature-login
```

WHY: Combine work

WHAT: Merges branch into current branch

WHEN: After feature is complete

👉 Happens mostly in `main` or `develop`

12. `git branch -d`

```
Bash
git branch -d feature-login
```

WHY: Clean up

WHAT: Deletes branch

👉 After merging

13. `git remote`

```
Bash
git remote -v
git remote add origin <url>
```

WHY: Connect to GitHub

WHAT: Manages remote repo links

👉 `origin` = default remote name

14. `git fetch`

```
Bash
git fetch
```

WHY: See updates without merging

WHAT: Downloads changes only

👉 Safe alternative to pull

15. `git reset`

```
Bash
git reset --soft HEAD~1
git reset --hard HEAD~1
```

WHY: Undo commits

WHAT:

- `--soft` → keep changes
- `--hard` → delete everything

👉 Dangerous but powerful

16. `git revert`

```
Bash
git revert <commit_id>
```

WHY: Undo safely

WHAT: Creates new commit that reverses changes

👉 Used in teams (safe option)

17. `git restore`

```
Bash
git restore file.txt
git restore --staged file.txt
```

WHY: Undo file changes

WHAT:

- restore file → discard changes
- restore staged → unstage

18. **git log**

```
Bash
git log --oneline
```

WHY: See history

WHAT: Shows commits

 Helps debug

19. **git diff**

```
Bash
git diff
```

WHY: See exact changes

WHAT: Line-by-line difference

20. **git stash**

```
Bash
git stash
git stash pop
```

WHY: Save work temporarily

WHAT: Hides changes

 Useful when switching branches mid-work

21. **git rebase**

```
Bash
git rebase main
```

WHY: Clean history

WHAT: Moves your commits on top of latest branch

👉 Advanced but important

22. **git cherry-pick**

```
Bash
git cherry-pick <commit_id>
```

WHY: Take specific commit

WHAT: Copies commit to current branch

23. **git tag**

```
Bash
git tag v1.0
```

WHY: Mark versions

WHAT: Creates release points

24. **git blame**

```
Bash
git blame file.txt
```

WHY: Find who wrote code

WHAT: Shows author per line

👉 Useful in debugging

🌟 REAL FLOW (Understand this deeply)

```
Bash
git init
git add .
git commit -m "first commit"
git branch feature
git checkout feature
git add .
git commit -m "feature work"
git checkout main
git merge feature
git push
```

🧠 Final Understanding

Git has **3 main areas**:

1. Working Directory (your files)
2. Staging Area (`git add`)
3. Repository (`git commit`)

👉 Most commands just move data between these 3.

If you want next level 🔥

I can:

- Give you **visual diagram of Git (easy brain understanding)**
- Simulate **real company workflow (with conflicts, PR, etc.)**
- Or debug your **actual Git errors step-by-step**

You said:

Give me a visual diagram of Git for easy brain understanding

ChatGPT said:



You said:

Give me a visual diagram of Git for easy brain understanding

ChatGPT said:

**You said:**

file create through terminal

ChatGPT said:

Got it—you want to **create files using the terminal (Git workflow style)**. Let's break it down simply and practically 🙌



1. Create File in Terminal

✓ Basic file creation

```
Bash
touch file.txt
```

WHY: Create an empty file

WHAT: Instantly makes a new file

👉 Most commonly used

✓ Create multiple files

```
Bash
touch index.html style.css script.js
```

2. Create + Write Content

Using **echo**

```
Bash
echo "Hello World" > file.txt
```

WHY: Quickly add content

WHAT: Creates file + writes text

Append content

```
Bash
echo "New line" >> file.txt
```

Using **cat**

```
Bash
cat > file.txt
```

Now type content → press **CTRL + D** to save

3. Edit File in Terminal

Using nano (beginner friendly)

```
Bash
nano file.txt
```

- Edit file
- **CTRL + X** → Save

Using vim (advanced)

```
Bash
vim file.txt
```

4. Create Folder

```
Bash
mkdir project
```

Create folder + file

```
Bash
mkdir project && cd project && touch app.js
```

5. Check Files

```
Bash
ls          # List files
ls -l       # Detailed view
```

6. Real Developer Flow (Important)

```
Bash
mkdir my-project
cd my-project

git init

touch index.html style.css script.js

git add .
git commit -m "Initial files"
```

 This is how real projects start.

Pro Tips

- **touch** = create file instantly
- **nano** = edit quickly
- **echo** = fast content

- Combine commands with **&&** for speed

If you want next level 🚀

I can:

- Show **how to create full project structure in 1 command**
- Teach **Linux commands every developer must know**
- Or give **real-world terminal practice tasks**

You said:

for windows

ChatGPT said:

On Windows, you can create files from the terminal using **Command Prompt (CMD)** or **PowerShell**. I'll show both clearly so you can use whichever you prefer.



1. Using Command Prompt (CMD)



Create empty file

```
bat
type nul > file.txt
```

WHY: Windows doesn't have **touch** like Linux

WHAT: Creates an empty file



Create file with content

```
bat
echo Hello World > file.txt
```

Append content

```
bat
echo New Line >> file.txt
```

Create folder

```
bat
mkdir project
cd project
```

Check files

```
bat
dir
```

2. Using PowerShell (Better Option)

Create empty file

```
PowerShell
New-Item file.txt
```

Shortcut:

```
PowerShell
ni file.txt
```

Create file with content

```
PowerShell
Set-Content file.txt "Hello World"
```

Append content

```
PowerShell
Add-Content file.txt "New Line"
```

Create folder

```
PowerShell
mkdir project
cd project
```

List files

```
PowerShell
ls
```

3. Best Option for Developers (Recommended)

Install **Git Bash** (comes with Git)

Then you can use Linux-style commands:

```
Bash
touch file.txt
nano file.txt
```

 Much easier + same as real dev environments

4. Real Developer Workflow (Windows)

```
Bash
mkdir my-project
cd my-project

git init

touch index.html style.css script.js # (Git Bash)

git add .
git commit -m "Initial commit"
```

Simple Understanding

Task	CMD	PowerShell	Git Bash
Create file	<code>type nul > file.txt</code>	<code>ni file.txt</code>	<code>touch file.txt</code>

Task	CMD	PowerShell	Git Bash
Write file	<code>echo ></code>	<code>Set-Content</code>	<code>echo ></code>
List files	<code>dir</code>	<code>ls</code>	<code>ls</code>



My Advice

Don't stick only to CMD. Real developers mostly use:

- **PowerShell** OR
- **Git Bash** (best for Git work)

If you want 🔥

I can:

- Show **how to install Git Bash + setup properly**
- Give you **20 terminal practice tasks (real developer level)**
- Or teach **advanced Windows terminal tricks**

You said:

git remote add origin <url> which url i use

ChatGPT said:

Good question—this is where many beginners get confused.



What URL to use in:

```
Bash
git remote add origin <url>
```

👉 You must use your **GitHub repository URL** (the repo you created on GitHub).



Where to find the URL

1. Go to your repo on GitHub

2. Click the green **"Code"** button

3. Copy one of these:

Option 1: HTTPS (Easy)

Example:

```
Bash
git remote add origin https://github.com/username/my-
project.git
```

WHY use this:

- Simple
- Works everywhere

👉 But may ask for username/token when pushing

Option 2: SSH (Best for developers)

Example:

```
Bash
git remote add origin git@github.com:username/my-project.git
```

WHY use this:

- No login every time
- More secure

👉 Requires SSH key setup

Check if added correctly

```
Bash
git remote -v
```

Full Example (Real Flow)

```
Bash
git init
git add .
git commit -m "first commit"

git remote add origin https://github.com/username/my-
project.git
git branch -M main
git push -u origin main
```

Common Mistakes

-  Using wrong repo URL
-  Copying browser URL (missing `.git`)
-  Repo not created yet on GitHub

Simple Rule

 **Whatever repo you see in your browser → copy its URL → use it here**

If you want next 
I can:

- Fix your **current Git error step-by-step**
- Help you **connect your exact repo (send screenshot)**
- Or teach **SSH setup (one-time, no password ever)**

You said:

git and github all command with meaning comment

ChatGPT said:

You don't need *every* obscure Git command—what you need is a **clean, real-developer command set with meaning**. Below is a **well-structured cheat sheet with comments (// meaning)** so you can understand and remember.

1. Setup (One-time)

```
Bash
git config --global user.name "Your Name"      # // set your
name
git config --global user.email "you@email.com" # // set your
email
git config --list                               # // check
config
```

2. Start / Get Repository

```
Bash
git init                # // start new Git repo in folder
git clone <url>         # // download repo from GitHub
```

3. Check Status

```
Bash
git status              # // show changed, staged, untracked
files
```

4. Add Files (Staging Area)

```
Bash
git add file.txt        # // stage specific file
git add .               # // stage all files
git add *.js            # // stage all JS files
```

5. Commit (Save Snapshot)

```
Bash
git commit -m "message" # // save staged changes
git commit -am "msg"    # // add + commit (tracked files
only)
```

6. Connect to GitHub (Remote)

```
Bash
git remote add origin <url>    # // connect local repo to
GitHub repo
git remote -v                  # // show remote URLs
```

7. Push & Pull

```
Bash
git push -u origin main    # // push code to GitHub (first
time)
git push                   # // push commits
git pull                   # // get latest changes + merge
git fetch                  # // get changes (no merge)
```

8. Branching (VERY IMPORTANT)

```
Bash
git branch                # // list branches
git branch feature        # // create branch
git checkout feature      # // switch branch
git switch feature        # // modern switch
git checkout -b feature   # // create + switch branch
```

9. Merge

```
Bash
git merge feature        # // merge feature into current
branch
```

10. Delete Branch

```
Bash
git branch -d feature    # // delete branch (after merge)
```

11. History & Changes

```
Bash
git log                # // full commit history
git log --oneline      # // short history
git diff               # // show file changes
git diff --staged      # // staged changes
```

12. Undo Changes

```
Bash
git restore file.txt    # // discard file changes
git restore --staged file.txt # // unstage file

git reset --soft HEAD~1 # // undo commit (keep changes)
git reset --hard HEAD~1 # // undo commit (delete changes)

git revert <commit_id>  # // safely undo commit
```

13. Stash (Temporary Save)

```
Bash
git stash          # // save changes temporarily
git stash list     # // show stash list
git stash pop      # // restore latest stash
```

14. Conflict Fix

```
Bash
git add .          # // after fixing conflict
git commit         # // finalize merge
```

15. Advanced Commands

```
Bash
git rebase main                # // move commits on top of main
git rebase -i HEAD~3          # // interactive rebase (edit
commits)

git cherry-pick <id>          # // apply specific commit

git tag v1.0                  # // mark version
git push origin v1.0          # // push tag
```

16. Authentication (HTTPS Token)

```
Bash
git remote set-url origin
https://<token>@github.com/user/repo.git # // use token
```

17. Clean Project

```
Bash
git clean -fd                # // remove untracked files/folders
```

18. Debug / Info

```
Bash
git show <id>                # // show commit details
git blame file                # // who changed each line
```

REAL DEVELOPER WORKFLOW (IMPORTANT)

```
Bash
git pull origin main          # // get latest code
git checkout -b feature-login # // create new feature
branch

git add .
git commit -m "Added login feature"

git push origin feature-login # // push feature branch
```

Core Concept (Most Important)

Git has **3 main areas**:

1. Working Directory → your files
2. Staging Area → `git add`
3. Repository → `git commit`

👉 Commands move data between these.

Golden Rules

- ✓ Always `git pull` before starting
- ✓ Use branches for every feature
- ✓ Never push directly to `main`
- ✓ Write clear commit messages
- ✓ Use `.gitignore`

If you want next level 🔥

I can:

- Give you **Git interview questions**
- Simulate **real company workflow (PR, merge, conflicts)**
- Or fix your **current GitHub push error step-by-step**

You said:

add delete command terminal

ChatGPT said:

Got it—you want **file/folder delete (and related) commands in terminal**, especially for Windows + Git workflow. I'll keep it clean with meaning 👉



1. Delete in Command Prompt (CMD)

✖ Delete file

```
bat
del file.txt      :: // delete a file
```

✖ Delete multiple files

```
bat
del *.txt         :: // delete all .txt files
```

✖ Delete folder

```
bat
rmdir folder      :: // delete empty folder
```

✖ Delete folder with files

```
bat
rmdir /s folder   :: // delete folder + all contents
```



Force delete without confirmation

```
bat
rmdir /s /q folder :: // delete silently (dangerous)
```



2. Delete in PowerShell

✖ Delete file

```
PowerShell
Remove-Item file.txt
```

✖ Delete folder

```
PowerShell  
Remove-Item folder
```

✖ Delete folder with contents

```
PowerShell  
Remove-Item folder -Recurse -Force
```

🐧 3. Delete in Git Bash (Linux style – recommended)

✖ Delete file

```
Bash  
rm file.txt
```

✖ Delete multiple files

```
Bash  
rm *.txt
```

✖ Delete folder

```
Bash  
rm -r folder
```

⚠ Force delete

```
Bash  
rm -rf folder # // delete everything without warning
```

👉 Be careful: **rm -rf** = no undo

4. Delete files using Git (IMPORTANT)

Remove file from Git + system

```
Bash
git rm file.txt      # // delete file and track deletion
```

Remove folder

```
Bash
git rm -r folder
```

Remove only from Git (keep locally)

```
Bash
git rm --cached file.txt  # // untrack but keep file
```

After deleting (important step)

```
Bash
git commit -m "deleted files"
git push
```

 Otherwise deletion won't reflect on GitHub

Simple Understanding

Action	Command
Delete file	<code>del / rm</code>
Delete folder	<code>rmdir / rm -r</code>
Force delete	<code>/q or -rf</code>
Delete in Git	<code>git rm</code>

Real Developer Tip

- Use normal delete (`rm`) for safety

- Avoid `rm -rf` unless you are 100% sure
- Always check using:

```
Bash  
git status
```

If you want next 🔥

I can:

- Give **50 terminal practice tasks (real dev level)**
- Teach **advanced file navigation commands (cd, mv, cp, etc.)**
- Or simulate a **real Git project workflow step-by-step**

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).